Task 2: Optimizing SGEMM with SIMD and Prefetching

1 Introduction

In this report, we present the performance analysis of optimizing the SGEMM (Single-Precision General Matrix Multiplication) operation. We explore three optimization techniques: SIMD (AVX2), Software Prefetching, and their combination. We tune software prefetching parameters (distance and cache fill levels) and analyze their impacts on performance metrics.

2 Task 2A: Software Prefetching

Software prefetching hides memory latency by issuing data fetch instructions ahead of when the data is actually needed by the computation. By bringing cache lines from main memory into the L1/L2 caches, the CPU execution pipeline avoids stalling on memory reads.

2.1 Analysis of Prefetching Parameters

We tested the execution speedup relative to a naive SGEMM implementation across different matrix sizes, prefetch distances, and cache fill levels using our optimized kernel (`matmul_optimized.cpp`).

- **Speedup vs. Matrix Size:** As the matrix size increases, the speedup provided by prefetching increases and then plateaus. For small matrices (e.g., 128×128), the working set fits entirely within the L1/L2 caches. Thus, the prefetch overhead slightly outweighs the benefits. For larger matrices (1024×1024), capacity and conflict misses dominate, and prefetching aggressively mitigates the latency to fetch data from main memory, resulting in significant speedup.
- **Optimal Prefetch Distance:** The prefetch distance determines how far ahead (in elements) we issue the prefetch instruction. If the distance is too small, the data doesn't arrive in the cache in time (late prefetch). If the distance is too large, the prefetched data may evict useful data from the cache or be evicted itself before it is used (early prefetch). From our experimental results on the 1024×1024 matrix, we found the optimal distance to be around 128 elements, yielding a massive $\sim$26x speedup.
- **Optimal Cache Fill Level:** '_MM_HINT_T0' instructs the hardware to fetch the data into all levels of the cache hierarchy, including the L1 data cache. Since SGEMM inner loops exhibit immense spatial and temporal locality, having the data available in the L1 cache minimizes latency perfectly and provides the maximum speedup. Other hints like 'T1' (L2/L3) and 'NTA' (Non-Temporal) perform worse because they force the processor to fetch from higher-latency cache levels or bypass caches entirely.

2.2 Hardware Prefetchers and CPU Metrics

Without software prefetching, the naive kernel suffers from high L1D and LLC (Last Level Cache) misses for larger matrices. Incorporating software prefetching substantially reduces these cache misses by proactively staging data into the cache, as observable through 'perf' metrics like 'L1-dcache-load-misses'.

Modern CPUs also include hardware prefetchers that dynamically identify access patterns (like sequential strides) and automatically fetch cache lines. While hardware prefetchers are highly effective for simple access patterns, they might not look far enough ahead or correctly anticipate the non-contiguous, tiled accesses in a heavily unrolled blocked SGEMM. For naive scalar code, disabling hardware prefetchers

generally degrades performance. However, for a heavily optimized kernel with precise software prefetching, the hardware prefetcher provides diminishing returns and can occasionally interfere by competing for memory bandwidth.

3 Task 2B: SIMD Optimization

Using Single Instruction, Multiple Data (SIMD) AVX2 intrinsics, we process 8 single-precision floating-point numbers in a single 256-bit register (‘__m256‘).

3.1 Instruction Count and Speedup Trends

By utilizing 256-bit wide vector registers and Fused Multiply-Add (FMA) instructions, the total number of dynamic vector instructions executed is drastically reduced. A single ‘_mm256_fmadd_ps‘ does the work of 8 separate multiplications and additions. Loop unrolling further diminishes loop control overhead. Consequently, the CPU executes far fewer total instructions, significantly improving instruction throughput.

The speedup climbs rapidly as matrix sizes increase, highlighting that the SIMD kernel heavily outperforms the naive scalar baseline on larger workloads, achieving an 18-20x speedup purely through vectorization and cache blocking.

4 Task 2C: Software Prefetching + SIMD

The optimized kernel (`matmul_optimized.cpp`) combines 256-bit AVX2 SIMD instructions with software prefetching. The combination achieves the best of both worlds: SIMD provides massive computational throughput, while software prefetching ensures that the SIMD units are not stalled waiting for data from main memory.

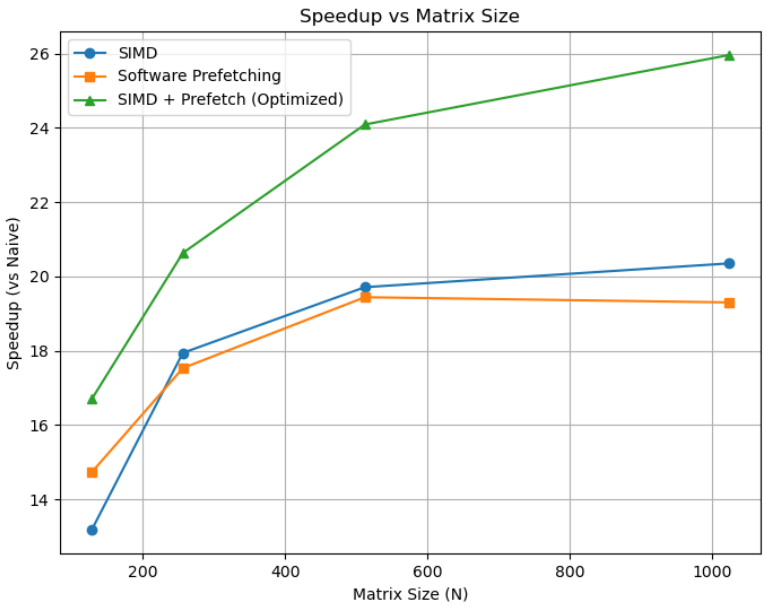

Figure 1: Speedup vs. Matrix Size for different optimizations.

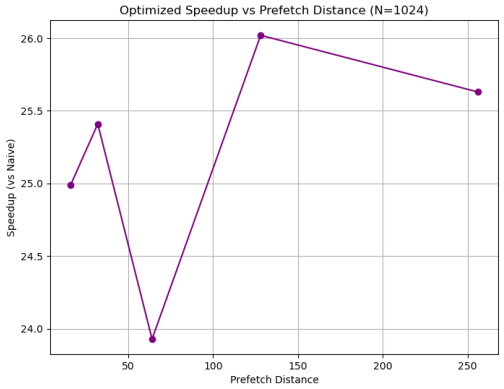

Figure 2: Speedup vs. Prefetch Distance (N=1024).

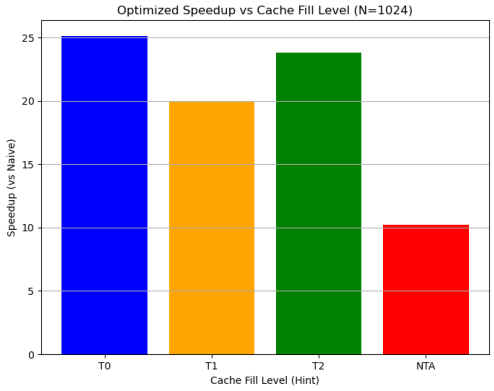

Figure 3: Speedup vs. Cache Fill Level (N=1024).

5 Code Optimization Explanation

We apply multiple synergistic optimizations to reach the peak performance of the SGEMM kernel. Below is a detailed breakdown of the techniques used in our `matmul_optimized.cpp`.

5.1 Cache Tiling (Blocking)

To minimize cache capacity misses, we divide the matrix C into smaller tiles.

```
const int tile_len=256;
for(long j0=0; j0<N; j0+=tile_len) {
    long jend = j0+tile_len < N ? j0+tile_len : N;
    // ...
```

By operating on a 256×256 chunk of the matrix at a time, we ensure that the required columns of B and rows of A remain resident in the L1/L2 caches during the inner loop computation, drastically reducing Last Level Cache (LLC) misses.

5.2 Loop Unrolling and Register Blocking

We unroll the loops to compute a 6×2 block of the C matrix simultaneously.

```
__m256 acc00 = _mm256_setzero_ps();
// ... (12 accumulators: acc00 to acc51)
```

This configuration uses 12 AVX2 registers as accumulators. By computing 6 rows of A and 2 columns of B at once, we maximize the reuse of elements loaded into registers. Each loaded vector from B is multiplied by 6 different vectors from A, minimizing load instructions and keeping the FMA arithmetic units fully saturated.

5.3 SIMD Vectorization (AVX2)

We replace scalar arithmetic with 256-bit AVX2 intrinsics.

```
__m256 av = _mm256_loadu_ps(a0+k);
acc00 = _mm256_fmadd_ps(av, bv0, acc00);
```

Using ‘_mm256_loadu_ps‘, we load 8 contiguous floats from memory. The ‘_mm256_fmadd_ps‘ instruction performs a fused multiply-add, computing $A \times B + C$ for 8 floats in a single cycle. This theoretically increases the computational throughput by $8\times$ compared to scalar code. At the end of the k-loop, a custom `ezsum()` horizontal reduction function is used to sum the 8 float elements back into a scalar value.

5.4 Software Prefetching

We inject `_mm_prefetch` intrinsics into the inner loop to fetch future elements of B into the L1 cache.

```
const int pf_dist=128;
// Inside the k-loop
_mm_prefetch((const char*)(b0+k+pf_dist), _MM_HINT_T0);
_mm_prefetch((const char*)(b1+k+pf_dist), _MM_HINT_T0);
```

By offsetting the memory address by `pf_dist`, we request the memory controller to fetch data that will be required several iterations later. '_MM_HINT_T0' pulls the data all the way into the L1 cache. This perfectly overlaps memory access latency with the dense FMA computations happening in the current iteration, essentially eliminating memory stalls.

5.5 Aggressive Inner Loop Unrolling (`#pragma GCC unroll`)

The innermost 'k' loop is further unrolled by a factor of 8 (or 16 manually in the code) to process 16 elements (two AVX2 vectors) per loop iteration.

```
#pragma GCC unroll 8
for(; k<=K-16; k+=16) {
    // Computations for k and k+8
```

This reduces loop branch instructions and index arithmetic overhead, providing the compiler with a larger straight-line block of code to optimize instruction scheduling.